

MoMo Analytics System

REST API, Authentication, DSA, and Testing

Report

Team: EMS Systems

Group Members: Erica Sheja, Ishimwe Samuel, Isingizwe Elohim Malaika

1. Introduction

The MoMo Analytics System processes Mobile Money SMS records provided in XML format and transforms them into structured transaction data. For this assignment, the project was extended with a REST API that allows authenticated clients to create, retrieve, update, and delete transaction records. The API was implemented in plain Python using the built-in **http.server** module. The project also includes a data structures and algorithms comparison between linear search and dictionary lookup.

The supplied dataset contains 1,691 parsed transaction records. The API reads these records from JSON and exposes them through HTTP endpoints. Basic Authentication protects all endpoints, as required by the assignment.

2. System and Data Flow

The project workflow combines the existing XML parsing and DSA work with the new API layer:

Stage	Description
XML data	Raw MoMo SMS records in XML format.
Parsing	SMS records are converted into structured JSON transaction objects.
Storage	The parsed records are stored in a JSON file containing 1,691 transactions.
API	The Python HTTP server exposes authenticated CRUD endpoints.
Client testing	curl commands were used to validate successful and unauthorized requests.
DSA comparison	We timed linear search and dictionary lookup over the transaction dataset.

3. REST API Implementation

The API is implemented in **api/server.py** using Python's **HTTPServer** and **BaseHTTPRequestHandler**. The API loads transaction data from **data/transactions.json** and saves it back to the same file when records are created, updated, or deleted.

Meth od	Endpoint	Purpose
GET	/transactions	Returns all transactions.
GET	/transactions/{id}	Returns one transaction by ID.
POST	/transactions	Creates a new transaction.
PUT	/transactions/{id}	Updates an existing transaction.
DELE TE	/transactions/{id}	Deletes a transaction.

3.1 Example API Requests

GET all transactions

```
curl -u admin: password http://localhost:8000/transactions
```

GET one transaction

```
curl -u admin: password http://localhost:8000/transactions/76662021700
```

POST a transaction

```
curl -u admin: password -X POST http://localhost:8000/transactions -H  
"Content-Type: application/json" -d '{"id": "TEST-001", "transaction_type":  
"payment", "amount_rwf":5000}'
```

MoMo Analytics System Page 2

PUT a transaction

```
curl -u admin: password -X PUT  
http://localhost:8000/transactions/TEST-001 -H "Content-Type:  
application/json" -d '{"transaction_type": "payment",  
"amount_rwf":7500}'
```

DELETE a transaction

```
curl -u admin: password -X DELETE  
http://localhost:8000/transactions/TEST-001
```

3.2 Response and Error Handling

The API returns JSON responses and uses HTTP status codes to communicate outcomes. Successful retrieval and update operations return 200 OK; creation returns 201 Created; invalid JSON or missing required information returns 400 Bad Request; missing or invalid authentication returns 401 Unauthorized;

unknown resources return 404 Not Found; and duplicate transaction IDs return 409 Conflict.

4. Authentication and Security

All five API operations check Basic Authentication before processing the request. The implementation reads the Authorization header, decodes the Base64 credentials, and compares the supplied username and password with the configured credentials. Requests without valid credentials return 401 Unauthorized and include a WWW-Authenticate header.

The testing confirmed that unauthenticated GET, POST, PUT, and DELETE requests were rejected. Authenticated requests were accepted, and the CRUD operations completed successfully.

4.1 Limitations of Basic Authentication

Basic Authentication is weak for production systems because it encodes the username and password with Base64 rather than encrypting them. Base64 can be decoded, so you should not transmit credentials over an unencrypted connection. HTTPS is therefore required when using Basic Authentication. The assignment implementation also stores the credentials directly in the Python source, which is suitable for demonstrating the required mechanism but should not be used for production secrets.

4.2 Stronger Alternatives

JWT: After successful authentication, a client can receive a signed JSON Web Token and use that token on later requests. This reduces the need to repeatedly send the username and password.

OAuth 2.0: OAuth 2.0 provides a framework for delegated authorization. It is useful when applications need controlled access to protected resources without directly sharing a user password with each client application.

5. DSA Comparison

The existing DSA implementation compares linear search with dictionary lookup for finding transactions by ID. The experiment used all 1,691 transactions and timed 10,000 searches for each method.

Method	Measured total time	Complexity
Linear search	0.896232 seconds	$O(n)$
Dictionary lookup	0.002017 seconds	$O(1)$ average case

```

apple@AppledeAir dsa % wc -l dsa_search.py
python3 dsa_search.py
  178 dsa_search.py
Loaded 1691 REAL transactions from /Users/apple/momo-analytics-system/dsa/modified_sms_v2.xml

Sanity check passed: both methods agree on every transaction id (and both return None for a missing id).

MoMo SMS API - DSA Comparison Report
=====

Data source: modified_sms_v2.xml (real parsed transactions)
Number of transactions searched over: 1691
Number of searches timed per method: 10000

Linear Search total time:    0.896232 seconds
Dictionary Lookup total time: 0.002017 seconds
Dictionary lookup was approximately 444.2x faster than linear search.

```

The recorded results show that dictionary lookup was approximately **444.2 times faster** than linear search in this experiment.

A linear search checks records one by one until it finds a matching ID, so the amount of work grows as the dataset grows. A dictionary uses a hash table, allowing an exact key lookup in $O(1)$ average time. The trade-off is additional memory and the cost of building the dictionary.

For other search requirements, such as range-based searches by amount or timestamp, a sorted structure with binary search can provide $O(\log n)$ search. A balanced tree or database B-tree index can also support efficient ordered and range queries.

Why is dictionary lookup faster than linear search?

A list has no idea where a specific id lives, so linear search has to check items one at a time until it finds a match. In the worst case it checks every single item. A dictionary works differently: it uses a hash table, which turns the key (the id) into a hash number and uses that number to jump almost directly to the right memory slot. A dictionary lookup therefore takes roughly the same time whether there are 20 transactions or 20k ($O(1)$, constant time), while linear search gets slower as the list grows ($O(n)$, linear time). The trade-off is that the dictionary uses extra memory and must be built once up front.

Can you suggest another data structure or algorithm that could improve search efficiency?

A dictionary is only fast for exact-key lookups. For searches such as "all transactions between amount X and amount Y" or "the next transaction after a certain timestamp", better options are:

- **Binary search on a sorted list:** if transactions are kept sorted by ID, amount, or date, binary search finds a record in $O(\log n)$ time by

repeatedly halving the search area. It is slower than a dictionary for exact-ID lookups, but it supports range queries that a dictionary cannot.

- **A balanced tree such as a B-Tree:** this is what most real databases use for indexes. It keeps data sorted and gives $O(\log n)$ lookups, a good middle ground between a plain list and a hash-based dictionary for large, frequently updated data like our momo transactions.

6. Testing and Validation

The API was tested using curl. The testing covered authenticated CRUD operations and unauthorized requests. A temporary transaction with ID TEST-001 was created, retrieved, updated from 5,000 RWF to 7,500 RWF, deleted, and then requested again to confirm that it no longer existed.

Test	Expected result	Observed
Authenticated GET	Transaction returned	Passed
Unauthorized GET	401 Unauthorized	Passed
POST	New transaction created	Passed
PUT	Existing transaction updated	Passed
DELETE	Transaction deleted	Passed
GET deleted ID	404 Transaction not found	Passed
Unauthorized POST	401 Unauthorized	Passed
Unauthorized PUT	401 Unauthorized	Passed
Unauthorized DELETE	401 Unauthorized	Passed

Screenshots of the required test cases are stored in the **screenshots/** directory of the repository.

7. Testing Evidence

Successful authenticated GET request

```
(.venv) user@Malaikas-Mac momo-analytics-system % curl -u admin:password http://localhost:8000/transactions/76662821700
{
  "id": "76662821700",
  "transaction_id": "76662821700",
  "transaction_type": "received",
  "amount_rwf": 2000,
  "timestamp": "10 May 2024 4:30:58 PM",
  "address": "M-Money",
  "body": "You have received 2000 RWF from Jane Smith (+++++013) on your mobile money account at 2024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662821700."
}
```

Unauthorized request returning 401

```
(.venv) user@Malaikas-Mac momo-analytics-system % curl http://localhost:8000/transactions
{"error": "Unauthorized", "message": "Valid username and password are required."}
```

Successful POST request

```
● (.venv) user@Malaikas-Mac momo-analytics-system % curl -u admin:password \
-X POST http://localhost:8000/transactions \
-H "Content-Type: application/json" \
-d '{
  "id": "SCREENSHOT-001",
  "transaction_id": "SCREENSHOT-001",
  "transaction_type": "payment",
  "amount_rwf": 5000,
  "timestamp": "29 September 2026 8:00:00 PM",
  "address": "M-Money",
  "body": "Screenshot test transaction"
}'
{
  "id": "SCREENSHOT-001",
  "transaction_id": "SCREENSHOT-001",
  "transaction_type": "payment",
  "amount_rwf": 5000,
  "timestamp": "29 September 2026 8:00:00 PM",
  "address": "M-Money",
  "body": "Screenshot test transaction"
}
○ (.venv) user@Malaikas-Mac momo-analytics-system %
```

Successful PUT request

```
● (.venv) user@Malaikas-Mac momo-analytics-system % curl -u admin:password \
-X PUT http://localhost:8000/transactions/SCREENSHOT-001 \
-H "Content-Type: application/json" \
-d '{
  "transaction_id": "SCREENSHOT-001",
  "transaction_type": "payment",
  "amount_rwf": 7500,
  "timestamp": "29 September 2026 8:00:00 PM",
  "address": "M-Money",
  "body": "Screenshot test transaction updated"
}'
{
  "transaction_id": "SCREENSHOT-001",
  "transaction_type": "payment",
  "amount_rwf": 7500,
  "timestamp": "29 September 2026 8:00:00 PM",
  "address": "M-Money",
  "body": "Screenshot test transaction updated",
  "id": "SCREENSHOT-001"
}
○ (.venv) user@Malaikas-Mac momo-analytics-system %
```

Successful DELETE request

```
● (.venv) user@Malaikas-Mac momo-analytics-system % curl -u admin:password \
-X DELETE http://localhost:8000/transactions/SCREENSHOT-001
{
  "message": "Transaction deleted successfully",
  "transaction": {
    "transaction_id": "SCREENSHOT-001",
    "transaction_type": "payment",
    "amount_rwf": 7500,
    "timestamp": "29 September 2026 8:00:00 PM",
    "address": "M-Money",
    "body": "Screenshot test transaction updated",
    "id": "SCREENSHOT-001"
  }
}
○ (.venv) user@Malaikas-Mac momo-analytics-system %
```

8. Conclusion

The assignment extension successfully adds a plain-Python REST API to the MoMo Analytics System. The API provides the required CRUD operations and protects every endpoint with Basic Authentication. Testing demonstrated successful authenticated requests and correct rejection of unauthorized requests. The DSA experiment also demonstrates the practical performance difference between linear search and dictionary lookup for exact transaction-ID searches.

For production use, manage authentication credentials through secure configuration rather than hard-coded source values; use HTTPS to protect network traffic; and consider stronger authentication or authorization mechanisms such as JWT-based authentication or OAuth 2.0.